Image Text Enhancer: A Lightweight, Easy-to-Use, and Efficient C++ Library

Algorithm Engineering 2026 Project

Daniel Motz
Friedrich Schiller University
Jena, Germany
daniel.motz@uni-jena.de

Leonard Teschner
Friedrich Schiller University
Jena, Germany
leonard.teschner@uni-jena.de

Mher Mnatsakanyan
Friedrich Schiller University
Jena, Germany
mher.mnatsakanyan@uni-jena.de

Abstract

TBC

Keywords

task parallelism, WebAssembly, OpenMP, image processing, C++

ACM Reference Format:

Daniel Motz, Leonard Teschner, and Mher Mnatsakanyan. 2026. Image Text Enhancer: A Lightweight, Easy-to-Use, and Efficient C++ Library: Algorithm Engineering 2026 Project. In *Proceedings of AEPRO 2026: Algorithm Engineering Projects (AEPRO 2026).* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

RQ Can a lightweight document enhancement pipeline be optimized to achieve strong multicore performance and practical WebAssembly deployment while preserving the enhancement behaviour of the baseline implementation?

RQ1. How much faster is the optimized pipeline than the baseline on native CPU execution?
RQ2. How well does the optimized pipeline scale under strong and weak scaling from 1 to 8 cores?
RQ3. What runtime overhead does the WebAssembly version introduce compared to native execution, and is that overhead still acceptable for practical use?

1.1 Background

The extraction of textual information from images — from scanned books to even receipts captured by smartphone cameras — have become an important part of everyday life [1, 8]. Any such extraction necessarily begins with the pre-processing of recorded images through techniques such as binarisation, noise removal, and geometric transformation, a process generally referred to as *document enhancement.* Numerous tools focused on this niche have been proposed. Although these approaches feature broad capabilities, they are often associated with substantial complexity on the user's end and entail considerable software dependencies. These dependencies restrict applicability, particularly in settings where binary size is constrained, such as mobile and web platforms. This limitation assumes added importance in a context where most digital media interactions now take place on smartphones and application development is increasingly oriented toward edge-computing models. Consequently, efficient and platform-independent implementations that exploit parallelization and other optimization techniques are of considerable importance for reducing processing time and enhancing user satisfaction.

Table 1: Comparison of related tools.

Tool	Lightweight	Licence	Commercial
ImageMagick	✗	Own	✓
Leptonica	✓	Own	✓
Unpaper	✗	Mixed	✗

1.2 Related Work

Image enhancement and document preprocessing are covered by a wide range of open-source tools and commercial software. The scope ranges from general purpose image processing and OCR libraries to specialized tools for post-processing scanned documents.

ImageMagick [4] is a suite for image processing that is available as a command line tool and as a library. It offers functions such as format conversion, resizing, filtering, and colour correction. However, ImageMagick is not specifically designed to enhance scanned or photographed images. Similar to OpenCV, the diverse features result in a large code base.

The C-based Leptonica library [3] is a general-purpose image processing and image analysis application. It is a dependency to both Tesseract and OpenCV. Although Leptonica is relatively lightweight compared to the other libraries, much like our approach, it primarily serves as a low-level library and does not offer ready-to-use, configurable processing pipelines.

A very similar tool to our approach is Unpaper [5], which focuses on post-processing scanned documents. It offers features such as deskewing, binarisation, and noise reduction that have been developed specifically for this purpose. The tool also offers a CLI variant, with which pipeline steps can be configured and disabled. Unpaper requires ffmpeg, which itself consumes >100MiB of space.

1.3 Our Contribution

Unlike the existing tools and libraries described, our approach is designed to be minimalistic, extensible, and user-friendly. Instead of being a comprehensive image processing library or OCR engine, we focus on a lightweight, modular image enhancement pipeline, focused on printed and handwritten text, written entirely in C++ with only CImg as a dependency, which is itself is lightweight has no further dependencies [7]. Image Text Enhancer (ITE) has a

lightweight build footprint, requiring only three build dependencies: OpenMP, X11, and JPEG. It can be imported as a library and also as a CLI executable—with 1.7 MiB on Linux.

We systematically optimize and improve our baseline implementation and evaluate the performance gains through various experiments. Compared to OpenCV and ImageMagick, our work is characterized by its deliberate focus on image enhancement and compact architecture. Unlike Tesseract and Leptonica, our pipeline is not integrated into an OCR workflow and limited by low-level primitive functions, but offers a configurable standalone solution that is ready for immediate use. Our pipeline is not exclusively specialized in the post-processing of scanned documents, but is designed as a general image enhancement pipeline that can be used for a variety of applications. In addition, our licensing model (MIT-0 and CeCILL-C) ensures its use in research and commercial applications. By combining minimal dependencies, a modular architecture, portability through C++ and OpenMP, and a commercially usable licence, our pipeline fills a gap between large-scale, general-purpose image processing libraries and highly specialized tools for document enhancement. In a proof of concept, we show that our approach can be executed on WASM platforms with only minor performance deficits.

1.4 Outline

In section 2, we introduce the pipeline that we will implement and optimize. In section 3, we describe the optimization techniques used. These are evaluated in section 4, where we compare the performance of the pipeline under different settings to the baseline implementation. Finally, we summarize the results in section **??** and provide an outlook on possible future work.

2 Pipeline

The pipeline is designed to be modular, enabling users to activate, deactivate and configure each stage as required, both through the CLI and the library. Some stages can intentionally not be disabled. These exceptions are marked in algorithm 1 with an **asterisk**.

Algorithm 1 Image Text Enhancer Pipeline

(1) Grayscale Conversion*
(2) Deskew
(3) Contrast Enhancement
(4) Denoise: (Adaptive) Gaussian and Median Blur
(5) Binarisation*: Otsu, Sauvola, Bataineh
(6) Morphological Operations
(7) Colour passthrough

The user can choose between different algorithms for certain stages—as these have different effects on the result, users can select the algorithm best suited to their application. For instance, Bataineh's adaptive binarisation method with automatic window adjustment [2] is helpful when binarising historical, handwritten documents with uneven writing and lighting, whereas Otsu's method [6] is a good choice for binarising typewritten documents with uniform font and lighting. Standard configurations are provided for each step, enabling the pipeline to be used straight away. These configurations deliver good results for most use cases and are based on fixed recommendations from the authors. This means that the pipeline can be applied immediately. However, users can also customise the configurations to optimise the pipeline and improve their specific results.

We provide a command-line application that can be used independently of a program to access the pipeline. This enables the pipeline to be configured and used directly via the command line.

3 Pipeline Optimizations

3.1 Parallelization and Concurrency

To achieve better performance on multicore processors, the individual steps are parallelized using OpenMP. *parallel for collapse(3)* environments are mainly used for the pixel processing loops. However, some binarisation algorithms also use other parallelization strategies such as the reduction clauses *reduction(min)* and *recuction(max)*. When implementing parallelization, care is taken to ensure that no race conditions or false sharing occur. For example, thread-local buffers are used for contrast enhancement to avoid false sharing and locking. In the deskewing logic, detection and application of the correction are separated. The angle is detected once and then applied to the grayscale and color image to avoid unnecessary calculations. Intuitively, this was done twice.

3.2 Memory Management and Allocation

To prevent increased memory consumption and long runtimes, the image is not copied at each step. Instead, the steps work in-place on the input image. For this purpose, a pointer is passed at each step. This approach is continued in grayscale conversion by replacing the final deep copy task of the image with a pointer swap of the original image pointer with the new one. Memory optimizations were made in further steps. For example, in the adaptive Gaussian blur filter and the adaptive median filter, the buffer allocations are removed from the pixel processing loop to reduce the number of memory allocations. Sauvola's binarisation algorithm uses a lightweight fused vector builder instead of the *8-bit CImg<double>* image structure. Similarly, the core utilities return preallocated buffers instead of heavy objects.

3.3 Algorithmic Complexity Reduction

Several different algorithms are implemented in the pipeline. Some of these are optimized through algorithmic improvements to reduce complexity. One example is Bataineh binarisation, which reduces the number of operations per pixel from $O(W^2)$ to $O(1)$ by using integral images to calculate the number of black and red pixels in the window instead of iterating over the entire window. In morphological operations, the complexity is reduced from $O(K^2)$ to $O(2K)$ by decomposing the 2D kernel operations into two 1D kernel operations for the horizontal and vertical dimensions. In addition, the sliding window min/max algorithm is implemented with a monotonic queue to enable computation for arbitrary window sizes in amortized $O(1)$ time. In the deskewing logic, the "rotate and project" method is replaced by a "Radon transform" method. Instead of moving the pixels, the coordinates of the foreground are extracted and mathematically projected.

3.4 Loop level Optimizations

To reduce the runtime of pixel processing loops, logical optimizations are made to the main loops that iterate over the pixels. In contrast enhancement, for example, the linear stretching formula is calculated in advance and stored in a lookup table. As a result, the main loop performs an $O(1)$ query per loop iteration instead of floating-point calculations. In addition, the if/else branch in the pixel processing loop is replaced by std::min clamping. This allows the compiler to auto-vectorize the loop with SIMD. In Sauvola binarisation, a branchless lookup is also implemented to improve performance.

3.5 Architectial Refactoring

In some cases, architectural changes were also made to improve performance. For example, in the deskewing logic, detection and application of the correction were separated. The angle is first detected and then applied to the grayscale and color images to avoid unnecessary calculations. In the core utilities, pre-built buffers are returned instead of creating heavy objects. This improves performance.

4 Experiments

4.1 Setup

To evaluate the performance of the pipeline with the optimizations, various experiments are conducted and the results are compared with the times of the intuitive implementation. The experiments examine the runtime when processing images of different sizes with and without parallelization.

The experiments assess three aspects: (1) single-threaded runtime versus image size, (2) eight-threaded runtime versus image size, and (3) strong-scaling behaviour with increasing core count for a large fixed image size.

For our measurements, we use a machine with an AMD Ryzen 9 9950X3D 16-core processor (32 hyperthreads) running Ubuntu 24.04.1 LTS with 64 GB of RAM. Each core has a base frequency of 4.3 GHz, with boost frequency of around 5.5 GHz. Care has been taken to limit the number of background processes, so as not to affect performance.

We selected a small but representative set of document images spanning both document type and resolution. The benchmark includes a small text page (1K), standard text documents at 2K, 4K, and 8K, a magazine-style page at 2K, and an extreme high-resolution text image at 16K. This choice captures common document-enhancement scenarios while also testing scalability to very large inputs. The set therefore reflects both everyday text-processing workloads and stress-test conditions for runtime evaluation.

4.2 Results

Figures **??** and 1 show the development of runtime with increasing image resolution for the intuitive and optimized implementations. It can be seen that runtime increases significantly with increasing image size, especially in the intuitive implementation. With the optimized implementation, the runtime is significantly lower, especially for larger images. At 16K resolution, the runtime of the intuitive implementation could no longer be calculated because small rounding errors in the deskewing step meant that the recognized angles in the color image and the grayscale image were no longer the same. As a result, the images could no longer be merged and the image could not be calculated. This problem is solved by the optimizations in the deskewing step, as the detection and application of the correction are separated here.

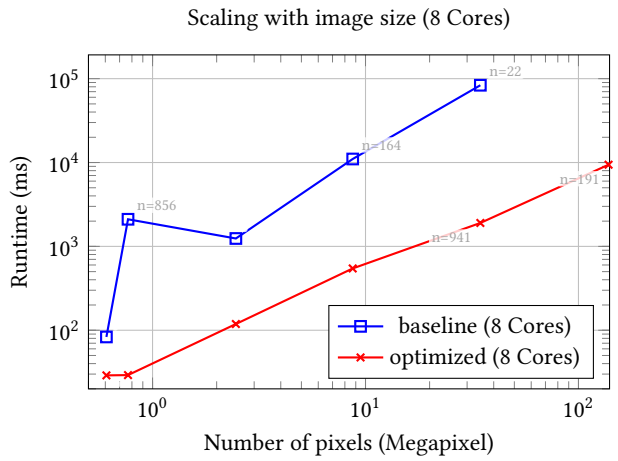

Figure 1: Runtime as a function of image size for the baseline and optimized implementations on 8 CPU cores. The x-axis shows the total image size in megapixels, and the y-axis shows the measured runtime in milliseconds; both axes use logarithmic scaling. Each plotted point corresponds to the mean runtime over repeated measurements for the respective input image. Labels are shown only for measurements whose sample count differs from the standard case of 1000 repetitions; in those cases, the label indicates the corresponding number of samples. For clarity, only one representative from the documents with image size of $\approx 10^1$ Megapixels is plotted, as the results were indistinguishable.

The results of the individual phases show a significant reduction in runtime in the deskewing and Bataineh binarisation phases when comparing the intuitive and optimized implementations. Table 3 shows the runtime of the individual phases for the KITA test image with a resolution of 3840x2254 on 8 cores. It can be seen that the optimizations lead to a massive reduction in phase runtime of over 98% for Bataineh binarisation in particular. The deskewing phase also benefits from the optimizations and shows a reduction in phase runtime of over 98%. In other phases, such as grayscale and the filter, a significant reduction in phase runtime can also be observed, while in the morphological ops phase, a slight increase in phase runtime can be seen. WHY?????. Overall, this leads to a reduction in the total runtime of over 95%. In absolute terms, the runtime improvement is mainly due to the optimization of Bataineh binarisation, as this phase accounts for the largest share of the total runtime in the intuitive implementation.

Table 4 shows the detailed phase runtime for the different phases and images. It can be seen that the optimizations lead to a significant

Table 2: Comparison of total runtimes (8 Cores)

Testbild	resolution	intuitive (ms)	optimized (ms)	Speedup
Text Small 1K	1017x597	130,577	42,511	3,07×
Text 2K	2048x1202	1.770,138	188,975	9,37×
Magazine 2K	2516x3272	16.701,170	649,924	25,70×
Text 4K	3840x2254	16.328,084	781,634	20,89×
Text 8K	7680x4508	114.607,067	3.015,428	38,01×
Text 16K	15360x9017	-*	15.181,484	-*

*Could not be calculated due to massive runtime

Table 3: Comparison of phase runtimes (3840x2254, 8 Cores)

Phase	intuitive (ms)	optimized (ms)	diff (%)
Init & Copy	65,458	63,672	-2,728%
Grayscale	30,950	17,753	-42,640%
Deskew	**159,405**	**3,062**	**-98,079%**
Filters (Sum)	483,312	380,467	-21,279%
Bataineh	**15.507,105**	**217,347**	**-98,598%**
Morphological Ops	66,663	93,05	+39,583%
Color Preservation	4,129	4,089	-0,969%
Total	**16.328,084**	**781,634**	**-95,213%**

reduction in runtime in all phases, especially in Bataineh binarisation, which accounts for the largest share of the total runtime in the intuitive implementation. For larger images such as the 8K text image, it can be seen that the image becomes too large for the system, as there is a sharp increase in the Init & Copy phase, which indicates a memory limitation.

5 Future Work

(1) Use heuristics for selecting algorithms, e.g. binarisation
(2) better deskew

References

[1] Musab Kasim Alqudah, Mohammad F. Bin Nasrudin, Bilal Bataineh, Mashal Alqudah, and Arwa Alkhatatneh. 2015. Investigation of binarization techniques for unevenly illuminated document images acquired via handheld cameras. In *2015 International Conference on Computer, Communications, and Control Technology (I4CT)*. 524–529. doi:10.1109/I4CT.2015.7219634

[2] Bilal Bataineh, Siti Abdullah, Khairuddin Omar, and Mohammad Faidzul Nasrudin. 2011. Adaptive Thresholding Methods for Documents Image Binarization, Vol. 6718. 230–239. doi:10.1007/978-3-642-21587-2_25

[3] Dan Bloomberg. 2001, 2024. Leptonica Image Processing Library. http://leptonica.org

[4] ImageMagick Studio LLC. 2024. *ImageMagick*. https://imagemagick.org

[5] Jens Ohrnberger. 2022. unpaper: Post-processing tool for scanned sheets. https://github.com/unpaper/unpaper

[6] Nobuyuki Otsu. 1979. A Threshold Selection Method from Gray-Level Histograms. *IEEE Transactions on Systems, Man, and Cybernetics* 9, 1 (1979), 62–66. doi:10.1109/TSMC.1979.4310076

[7] Jean-Philippe Tarel. 2024. CImg: A Simple C++ Toolkit for Image Processing. https://cimg.eu. Version 3.x.

[8] Yanxi Zhou, Shikai Zuo, Zhengxian Yang, Jinlong He, Jianwen Shi, and Rui Zhang. 2023. A Review of Document Image Enhancement Based on Document Degradation Problem. *Applied Sciences* 13, 13 (2023). doi:10.3390/app13137855

Table 4: Detaillierte Laufzeit der Algorithmus-Phasen in Millisekunden (Auszug)

Phase	1K Small	2K Text	2K Mag.	4K Text	8K Text
Init & Copy	1,29	4,20	61,83	65,46	241,93
Grayscale	0,60	2,95	27,68	30,95	92,45
Deskew	24,22	59,89	28,90	159,41	518,54
Binarisation	71,00	1.555,57	15.999,83	15.507,11	111.495,99
Filter (Sum)	28,74	125,14	520,73	483,31	1.968,43
TOTAL	**130,58**	**1.770,14**	**16.701,17**	**16.328,08**	**114.607,07**